Virtualization

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercises

Practical Lab: Exploring Virtualization & Emulation

This guide will walk you through different forms of virtualization, from lightweight emulation to full-blown server management. You will use **VirtualBox** (for Windows/macOS) or **QEMU** (for Linux) as your primary tool.

> **Before you begin:** Make sure you have at least **50 GB of free disk space** and a stable internet connection. Some downloads are large and VM disk images can grow quickly.

Part 1: Host Setup – Your Virtualization Tool

First, install the correct tool for your operating system.

For Windows & macOS Hosts: VirtualBox

1. **Download the Installer:**
   - Go to the VirtualBox downloads page and download the installer for your OS (Windows or macOS).
   - Also, download the **VirtualBox Extension Pack** from the same page (it is a single file that works for all platforms).
2. **Install VirtualBox:**
   - **Windows:** Run the `.exe` installer. Accept the default options. Windows may ask you to approve network driver installation — click **Yes**.
   - **macOS:** Open the `.dmg` file and run the installer. You **must** go to `System Settings > Privacy & Security` and click **Allow** to approve the Oracle system extension. You may need to reboot.
3. **Install the Extension Pack:**
   - Open VirtualBox. Go to **File > Tools > Extension Pack Manager** (or **Preferences > Extensions** in older versions).
   - Click the **Install** icon and select the Extension Pack file you downloaded.
   - Accept the license agreement.
4. **Verify the Installation:**
   - Open VirtualBox. You should see the main manager window with an empty VM list.
   - Go to **Help > About VirtualBox** and confirm the version number matches the Extension Pack version.
   - If you see any warnings about kernel drivers or modules, follow the on-screen instructions before proceeding.

For Linux Hosts: QEMU/KVM

1. **Install Packages:**
   - On Debian/Ubuntu, open a terminal and run:
     ```
     $ sudo apt update
     $ sudo apt install qemu-system-x86 qemu-system-i386 qemu-utils bridge-utils
     ```
   - On Fedora:

```
$ sudo dnf install qemu-system-x86 qemu-img bridge-utils
```

2. **Enable KVM Access:**
   - Add your user to the `kvm` group so you can run VMs without sudo:
     ```
     $ sudo usermod -a -G kvm $USER
     ```
   - **Important:** You must log out and log back in (or reboot) for the group change to take effect.
3. **Verify the Installation:**
   - Check that KVM is available:
     ```
     $ kvm-ok
     ```
     You should see: `KVM acceleration can be used`. If not, you may need to enable Intel VT-x or AMD-V in your BIOS/UEFI settings.
   - Confirm QEMU is installed:
     ```
     $ qemu-system-x86_64 --version
     ```
     This should print the QEMU version number.

Preparing for cloud-init automation: To use cloud-init for automated VM setup (Debian, Ubuntu, etc.), you need the `cloud-localds` utility:

- **Debian/Ubuntu:**

```
sudo apt update
sudo apt install cloud-utils
```

This provides the `cloud-localds` tool to create a cloud-init "seed" ISO from your YAML configuration files. *(You only need this if doing cloud-init automation; skip if doing all setup manually inside the VM.)*

Part 2: Lightweight Emulation with FreeDOS

In this exercise we explore a simple, single-tasking operating system (FreeDOS) to understand basic machine emulation. FreeDOS is an open-source implementation of MS-DOS and can run classic DOS software, including games.

> **Important Boot Sequence Note:** You must boot your VM twice for FreeDOS installation: 1. The **first boot** uses the FreeDOS ISO. Select "Install to harddisk," partition/format, and install packages as instructed. 2. When installation completes and you are prompted to reboot, **remove the ISO from your virtual drive before rebooting**. 3. The **second boot** will start from the virtual hard disk you just installed FreeDOS on. If you forget to remove the ISO, you may inadvertently restart the installer instead of booting into your newly-installed OS!
>
> **Tip:** After verifying FreeDOS boots from the hard disk, **power down the VM**. Replace the FreeDOS install ISO (in your virtual CD drive) with the ISO containing the DOOM game files (`doom.iso`). Then start the VM again; the new ISO will appear as a CD drive (usually D:), allowing you to transfer game files.

Step 1 – Download Resources

1. Download the **FreeDOS 1.4 Live CD** from the official site. You need the `FD14-LiveCD.zip` file.
2. Unzip the archive. Inside you will find the file `FD14LIVE.iso`.
3. Download the shareware version of **DOOM** (`doom19s.zip`) from this archive.
4. Unzip `doom19s.zip` into a folder called `doom/` on your host machine.

Step 2 – Create the FreeDOS Virtual Machine VirtualBox:

1. Open VirtualBox and click **New**.
2. Configure the VM:
   - **Name:** `FreeDOS`
   - **Type:** `Other`
   - **Version:** `DOS`
3. Set **Memory** to `64 MB`. (DOS does not need more.)
4. For the **Hard Disk**, choose *Create a virtual hard disk now*:
   - **File type:** VDI
   - **Size:** `512 MB`
   - Choose **Fixed size** for better performance.
5. Click **Create** to finish the wizard.

6. With the new FreeDOS VM selected, click **Settings > Storage**.
7. Under **Controller: IDE**, click the **Empty** disc icon.
8. On the right side, click the small CD icon and choose **Choose a disk file...**.
9. Select the `FD14LIVE.iso` file you extracted earlier.
10. Click **OK** to save.

QEMU (Linux):

1. Create a 500 MB disk image:

   ```
   $ qemu-img create -f qcow2 freedos.qcow2 500M
   ```

2. Launch the VM with the Live CD attached:

   ```
   $ qemu-system-i386 -machine accel=kvm:tcg -m 128 -cpu host \
       -k pt -rtc base=localtime \
       -device adlib -device sb16 \
       -device cirrus-vga -display gtk \
       -hda freedos.qcow2 \
       -cdrom FD14LIVE.iso -boot d
   ```

   Note: The `-device adlib -device sb16` flags emulate classic sound cards so DOS games can produce audio.

Step 3 – Install FreeDOS

1. Boot the VM. You will see the FreeDOS boot menu.
2. Select **Install to harddisk**.
3. Follow the on-screen prompts:
   - When asked to partition the disk, accept the default (use the whole disk for `C:`).
   - When asked to format, confirm **Yes** (FAT32 format).
   - Select the packages you want to install (the defaults are fine).
4. Wait for the installation to complete. It will ask you to reboot.
5. **Before rebooting, remove the ISO:**
   - **VirtualBox:** Go to **Settings > Storage**, click the ISO under the IDE controller, click the CD icon on the right, and choose **Remove Disk from Virtual Drive**. Then reboot the VM.
   - **QEMU:** Close the QEMU window. Re-launch without `-cdrom` and `-boot d`:
     ```
     $ qemu-system-i386 -machine accel=kvm:tcg -m 128 -cpu host \
         -k pt -rtc base=localtime \
         -device adlib -device sb16 \
         -device cirrus-vga -display gtk \
         -hda freedos.qcow2 -boot c
     ```
6. **Verify:** The VM should boot into FreeDOS from the hard disk and show the `C:\>` prompt.

Step 4 – Transfer the Game into the VM Since FreeDOS has no network stack by default, we will use a virtual drive to move files.

VirtualBox:

1. Use a free tool such as **AnyBurn** (Windows) or **Brasero** (Linux) to create an ISO file from your `doom/` folder. Name it `doom.iso`.
2. In VirtualBox, go to **Settings > Storage** for the FreeDOS VM.
3. Click the **Add Optical Drive** icon on the IDE Controller, then select your `doom.iso`.
4. Start the VM. The ISO will appear as drive `D:`.

QEMU (Linux):

QEMU can expose a host folder as a virtual FAT drive. Add this flag when launching the VM:

```
$ qemu-system-i386 -machine accel=kvm:tcg -m 128 -cpu host \
    -k pt -rtc base=localtime \
    -device adlib -device sb16 \
    -device cirrus-vga -display gtk \
    -hda freedos.qcow2 -boot c \
    -drive file=fat:rw:doom/,format=raw
```

The doom/ folder contents will appear as drive D: inside FreeDOS.

Step 5 – Install and Run the Game

1. At the FreeDOS prompt, switch to the game drive:

   ```
   C:\> D:
   D:\> dir
   ```

   You should see the DOOM files listed.

2. Copy the files to the hard disk (optional but recommended):

   ```
   D:\> mkdir C:\DOOM
   D:\> copy *.* C:\DOOM
   ```

3. Run the game:

   ```
   D:\> DOOM.EXE
   ```

   Or, if you copied it:

   ```
   C:\> cd DOOM
   C:\DOOM> DOOM.EXE
   ```

4. **Verify:** You should see the DOOM title screen and hear sound effects through the emulated sound cards. Use the arrow keys and Ctrl to play.

Step 6 – Reflect Answer these questions in your lab notes:

- What type of virtualization is being used here (emulation, full virtualization, or paravirtualization)?
- Why does FreeDOS only need 64 MB of RAM?
- What is the role of the -device sb16 flag? What happens if you remove it?

Part 3: Lightweight Virtualization with Alpine Linux

Alpine Linux is a security-oriented, lightweight Linux distribution. It is widely used as the base image for Docker containers. In this exercise you will install it in a VM, explore networking modes, and set up a web server.

Step 1 – Download the ISO

1. Go to the Alpine Linux downloads page.
2. Download the **Standard** image for your architecture:
   - Most PCs: x86_64
   - Apple Silicon Macs: aarch64
3. Note the downloaded filename (e.g., alpine-standard-3.22.1-x86_64.iso).

Step 2 – Create the Alpine VM VirtualBox:

1. Open VirtualBox and click **New**.
2. Configure the VM:
   - **Name:** Alpine
   - **Type:** Linux
   - **Version:** Other Linux (64-bit)
3. Set **Memory** to 1024 MB (1 GB).
4. For the **Hard Disk**, create a new VDI with **8 GB** (dynamically allocated is fine).
5. Click **Create**.
6. Go to **Settings > Storage** and attach the Alpine ISO to the empty CD drive (just like you did for FreeDOS). 228: 7. Click **OK**.

   Cloud-init Automation for Alpine Linux (Optional/Advanced): Alpine Linux supports cloud-init automation using a "config drive" seeded with your configuration. 1. Use the cloud-localds tool to create a seed ISO containing your user-data and meta-data YAML files: bash cloud-localds seed.iso user-data meta-data 2. In VirtualBox, attach this

seed ISO as a second CD-ROM drive before first Alpine boot. 3. On first boot, Alpine's cloud-init (if included) will discover and apply these settings. For examples and details, see Alpine cloud-init documentation.

QEMU (Linux):

1. Create an 8 GB disk image:

   ```
   $ qemu-img create -f qcow2 alpine.qcow2 8G
   ```

2. Launch the VM with the ISO:

   ```
   $ qemu-system-x86_64 -machine q35,accel=kvm:tcg \
       -m 1G -smp 2 -cpu host \
       -k pt -rtc base=localtime -display gtk \
       -drive file=alpine.qcow2,format=qcow2,if=virtio \
       -cdrom alpine-standard-3.22.1-x86_64.iso -boot d \
       -nic user,model=virtio-net-pci,hostfwd=tcp::2222-:22,hostfwd=tcp::8080-:80
   ```

   Note: The `-nic user,...,hostfwd=tcp::2222-:22,hostfwd=tcp::8080-:80` flag sets up NAT networking and forwards host port 2222 to guest port 22 (SSH) and host port 8080 to guest port 80 (HTTP).

Step 3 – Install Alpine Linux

1. Boot the VM. After a few seconds you will see a login prompt.
2. Log in as `root` (no password required on the live image).
3. Run the installer:

   ```
   # setup-alpine
   ```

4. Follow the prompts carefully:

   - **Keyboard layout:** Choose your layout (e.g., `pt` for Portuguese, `us` for US English).
   - **Hostname:** Accept the default (`alpine`) or choose your own.
   - **Network:** Select the detected interface (usually `eth0`). Choose `dhcp` for automatic IP.
   - **Root password:** Set a password you will remember (e.g., `student`).
   - **Timezone:** Choose your timezone (e.g., `Europe/Lisbon`).
   - **Proxy:** Press Enter to skip (none).
   - **Mirror:** Type a number or press `f` to auto-detect the fastest mirror.
   - **SSH server:** Choose `openssh`.
   - **Disk:** Type `sda` (or `vda` if using virtio).
   - **Install type:** Choose `sys` (full install to disk).
   - **Erase disk:** Confirm with `y`.

5. Wait for the installation to finish.
6. When done, type `poweroff` to shut down the VM.
7. **Remove the ISO:**

   - **VirtualBox:** Go to **Settings > Storage**, select the ISO, and remove it from the drive.
   - **QEMU:** Re-launch without `-cdrom` and `-boot d` flags.

Step 4 – Boot and Verify

1. Start the VM again (without the ISO).
2. Log in as `root` with the password you set.
3. Verify network connectivity:

   ```
   # ip addr show
   # ping -c 3 google.com
   ```

4. **Verify from your host:** Open a terminal on your host machine and try:

   - **QEMU users:** `ssh root@localhost -p 2222`
   - **VirtualBox NAT users:** You need to set up port forwarding first (see Step 5).

Step 5 – Explore Networking Modes Understanding NAT (the default):

With NAT, the VM can access the internet through the host, but the host cannot directly reach the VM. The VM gets a private IP (usually `10.0.2.15`).

To reach the VM from the host in NAT mode, you must configure **port forwarding**:

- **VirtualBox:** Go to **Settings > Network > Advanced > Port Forwarding**. Add two rules:

Name	Protocol	Host Port	Guest Port
SSH	TCP	2222	22
HTTP	TCP	8080	80

  Now from your host you can run:

  ```
  $ ssh root@localhost -p 2222
  ```

- **QEMU:** Port forwarding was already configured in the launch command with `hostfwd=tcp::2222-:22`.

Switching to Bridged Mode:

With a bridged network, the VM gets its own IP address from your local network (e.g., `192.168.1.x`), as if it were another physical computer on the network.

1. Shut down the VM.
2. **VirtualBox:** Go to **Settings > Network**. Change **Attached to:** from `NAT` to `Bridged Adapter`. Select your host's network interface from the dropdown.
3. **QEMU:** This requires creating a network bridge on your host, which needs root access. See the provided `alpine.sh` script in the support materials for a working example.
4. Start the VM and check the new IP:

   ```
   # ip addr show eth0
   ```

   You should see an IP from your local network (e.g., `192.168.1.123`).
5. From your host, you can now reach the VM directly:

   ```
   $ ssh root@192.168.1.123
   ```

Verify the difference: Note how with NAT you needed port forwarding (`localhost:2222`), but with bridged mode you connect directly to the VM's IP.

Step 6 – Set Up a Web Server

1. Make sure the VM is running and you are logged in as `root`.
2. Update the package index and install a lightweight web server:

   ```
   # apk update
   # apk add lighttpd
   ```

3. Create a simple HTML page:

   ```
   # mkdir -p /var/www/localhost/htdocs
   # cat > /var/www/localhost/htdocs/index.html << 'EOF'
   <!DOCTYPE html>
   <html>
   <head><title>Alpine VM</title></head>
   <body>
     <h1>Hello from Alpine Linux!</h1>
     <p>This page is served from a virtual machine.</p>
   </body>
   </html>
   EOF
   ```

4. Start the web server:

   ```
   # rc-service lighttpd start
   ```

5. Verify inside the VM:

   ```
   # wget -qO- http://localhost
   ```

   You should see the HTML content you just created.

6. **Access from your host machine:**
   - **NAT mode (VirtualBox with port forwarding or QEMU):** Open your web browser and navigate to `http://localhost:8080`.
   - **Bridged mode:** Navigate to `http://<VM_IP>` (e.g., `http://192.168.1.123`).
7. **Verify:** You should see the "Hello from Alpine Linux!" page in your browser.

Step 7 – Enable the Web Server at Boot (Optional) To make the web server start automatically when the VM boots:

```
# rc-update add lighttpd default
```

Reboot the VM and verify the page is still accessible without manually starting the service.

Step 8 – Reflect Answer these questions in your lab notes:

- What type of virtualization is Alpine running under (emulation or full virtualization)?
- What is the difference between NAT and Bridged networking? When would you use each?
- Why is Alpine Linux so popular for containers and VMs?
- What is the size of the Alpine ISO compared to a typical Ubuntu or Windows ISO?

Part 4: Server Management with Proxmox VE

Proxmox Virtual Environment (VE) is a professional, open-source server virtualization platform. It combines KVM (for VMs) and LXC (for containers) with a web-based management interface. In this exercise you will install Proxmox inside a VM to explore its capabilities.

> **Caution: Nested Virtualization.** You will run a hypervisor (Proxmox) inside another hypervisor (VirtualBox/QEMU). This is called **nested virtualization**. It is resource-intensive and will be slow. This exercise is for learning purposes.

Step 1 – Download Proxmox

1. Go to the Proxmox VE downloads page.
2. Download the latest **Proxmox VE ISO Installer** (it will be approximately 1.2 GB).

Step 2 – Create the Proxmox VM This VM needs significantly more resources than the previous exercises.

VirtualBox:

1. Click **New** and configure:
   - **Name:** `Proxmox`
   - **Type:** `Linux`
   - **Version:** `Debian (64-bit)`
2. Set **Memory** to `4096 MB` (4 GB) or more.
3. Set **Processors** to 2 or more.
4. Create a hard disk of at least **32 GB** (dynamically allocated).
5. Before starting, go to **Settings** and apply these changes:
   - **System > Processor:** Check **Enable Nested VT-x/AMD-V** (this allows Proxmox to run VMs inside itself).
   - **Network > Adapter 1:** Set **Attached to:** NAT.
   - **Network > Adapter 1 > Advanced > Port Forwarding:** Add a rule:

Name	Protocol	Host Port	Guest Port
WebUI	TCP	8006	8006

   - **Storage:** Attach the Proxmox ISO to the CD drive.
6. Click **OK** to save all settings.

QEMU (Linux):

1. Create a 32 GB disk image:

   ```
   $ qemu-img create -f qcow2 proxmox.qcow2 32G
   ```

2. Launch the VM for installation:

   ```
   $ qemu-system-x86_64 -machine q35,accel=kvm:tcg \
       -m 4G -smp 2 -cpu host \
       -k pt -rtc base=localtime -display gtk \
       -drive file=proxmox.qcow2,format=qcow2,if=virtio \
       -cdrom proxmox-ve.iso -boot d \
       -nic user,model=virtio-net-pci,hostfwd=tcp::8006-:8006
   ```

   Note: The `-cpu host` flag is critical. It passes the host CPU's virtualization capabilities (VT-x/AMD-V) to the guest, which Proxmox needs to create nested VMs.

Step 3 – Install Proxmox

1. Boot the VM. The Proxmox graphical installer will start.
2. Click **Install Proxmox VE**.
3. Accept the license agreement (EULA).
4. Select the target hard disk (the only option should be your virtual disk).
5. Set your country, timezone, and keyboard layout.
6. Set the **root password** (remember it!) and enter an email address (can be fictional for lab purposes, e.g., `admin@lab.local`).
7. **Network configuration:**
   - **Hostname:** `proxmox.lab.local`
   - **IP Address:** Use the address suggested by the installer (usually `10.0.2.15/24` in NAT mode).
   - **Gateway:** `10.0.2.2` (default QEMU/VirtualBox NAT gateway).
   - **DNS:** `10.0.2.3` or `8.8.8.8`.
8. Review the summary and click **Install**.
9. Wait for the installation to complete (5-10 minutes).
10. When prompted, click **Reboot**.
11. **Remove the ISO** from the virtual drive before the reboot completes.

Step 4 – Access the Proxmox Web Interface

1. After reboot, the Proxmox console will display a URL like:

   `https://10.0.2.15:8006/`

2. Since we configured port forwarding, open your **host machine's web browser** and navigate to:

   `https://localhost:8006`

3. You will see a security warning about the self-signed certificate. This is expected — click **Advanced** and then **Proceed** (or **Accept the Risk**).

4. Log in with:

   - **Username:** `root`
   - **Realm:** `Linux PAM standard authentication`
   - **Password:** the password you set during installation.

5. **Verify:** You should see the Proxmox dashboard with your node listed on the left side, showing CPU usage, memory, and storage.

Step 5 – Explore the Proxmox Interface Take a few minutes to explore:

1. Click on your node name (e.g., `proxmox`) in the left sidebar.
2. Examine the **Summary** tab: CPU usage, RAM, uptime.
3. Go to **Datacenter > Storage**: See the default storage (`local` and `local-lvm`).
4. Go to **Datacenter > Network**: See the configured network interfaces.

Step 6 – Create a Guest VM inside Proxmox (Challenge) This is a stretch goal. Try to create an Alpine Linux VM inside Proxmox:

1. **Upload the Alpine ISO:** In the Proxmox web interface, go to your node > **local (storage)** > **ISO Images** > **Upload**. Upload the Alpine ISO you downloaded earlier.
2. **Create a VM:** Click the **Create VM** button in the top-right corner.
   - **General:** Give it a name (e.g., `alpine-nested`).
   - **OS:** Select the uploaded Alpine ISO.
   - **System:** Accept defaults.
   - **Disks:** Set disk size to `2 GB`.
   - **CPU:** `1 core`.
   - **Memory:** `512 MB`.
3. Click **Finish**, then select the new VM and click **Start**.
4. Open the **Console** tab to see the Alpine boot process.
5. **Verify:** Can you log in as `root` on the nested Alpine VM? Can it reach the internet?

   Note: Nested VMs will be very slow because of double virtualization overhead. This is expected behavior.

Step 7 – Reflect Answer these questions in your lab notes:

- What is the difference between a Type 1 and Type 2 hypervisor? Which type is Proxmox?
- Why did we need to enable "Nested VT-x/AMD-V" in VirtualBox?
- What advantages does a web-based management interface provide over command-line tools?
- Could you use Proxmox in a production environment? What would be different from this lab setup?

Part 5: Bonus – Emulating Android

The best way to emulate Android on a PC is using the official tools from Google.

Step 1 – Install Android Studio

1. Go to the Android Studio download page.
2. Download and install the version for your OS.
3. The installer will download additional components (SDK, emulator images). This may take **15-30 minutes** depending on your connection.

Step 2 – Create a Virtual Device

1. Open Android Studio. You do **not** need to create a project.
2. From the welcome screen, click **More Actions > Virtual Device Manager** (or from the **Tools** menu if you have a project open).
3. Click **Create Virtual Device**.
4. Choose a phone hardware profile (e.g., **Pixel 7**) and click **Next**.
5. Select a system image to download:
   - Choose a recent API level (e.g., API 34, Android 14).
   - Click **Download** next to the image name and wait for it to complete.
   - Select the downloaded image and click **Next**.
6. Give your AVD (Android Virtual Device) a name and click **Finish**.

Step 3 – Launch and Explore

1. In the Virtual Device Manager, click the **Play** button next to your device.
2. A new window will open showing the Android boot animation.
3. Once booted, explore:
   - Open the **Settings** app and look at device info.
   - Open the **Chrome** browser and visit a website.
   - Open the **Files** app and explore the virtual file system.
4. **Verify:** The emulated phone should behave exactly like a real Android device, including touch input (via mouse clicks), rotation, and GPS simulation.

Step 4 – Reflect Answer these questions in your lab notes:

- What type of virtualization does the Android emulator use? Is it emulation, full virtualization, or something else?
- How does the Android emulator achieve near-native performance on x86 hosts?
- What is the role of the Android SDK in this setup?

Summary and Deliverables

By the end of this lab, you should have:

1. A working virtualization tool (VirtualBox or QEMU) installed on your host.
2. A FreeDOS VM capable of running a classic DOS game.
3. An Alpine Linux VM with network connectivity and a running web server accessible from your host.
4. A Proxmox VM with a working web interface (and optionally, a nested VM inside it).
5. (Bonus) An Android virtual device running in the Android emulator.

Submit your lab notes with the answers to all reflection questions and screenshots showing:

- The FreeDOS VM running DOOM.
- The Alpine web server page displayed in your host browser.
- The Proxmox web interface dashboard.